

Relatório de Evolução — EV Challenge GoodWe

Sprint 03 — AsterCharge

1. Resumo da evolução

Nas Sprints 1 e 2, o grupo desenvolveu um chatbot de atendimento para os eletropostos da ChargeGrid. A Sprint 1 definiu o problema, a persona do chatbot, o system prompt e um conjunto de 5 perguntas de teste. A Sprint 2 transformou esse projeto em uma aplicação funcional: um loop manual em Python, usando a API da OpenAI (modelo gpt-4o-mini) e uma lista simples de mensagens para guardar o histórico da conversa.

Na Sprint 03, essa arquitetura foi substituída por uma baseada em agentes de IA, usando o OpenAI Agents SDK. O chatbot deixou de ser só uma chamada direta ao modelo e passou a contar com ferramentas (tools) para consultar dados de recarga, memória de sessão gerenciada pelo próprio framework, e guardrails de segurança que analisam tanto a entrada quanto a saída das mensagens. Também foi testado um segundo modelo de linguagem para comparação.

2. Refatoração

A principal decisão foi adotar o OpenAI Agents SDK, já que nós já utilizávamos a API da OpenAI desde a Sprint 2. A mudança aproveitou esse conhecimento e trocou só a forma de organizar a conversa, sem trocar de provedor. O núcleo do agente foi reorganizado em quatro partes: as tools, os guardrails, o agente principal e a memória de sessão.

O maior trade-off encontrado foi entre “resposta rápida e genérica” e “resposta precisa baseada em dados”: ao dar ao agente acesso às tools, ele passou a pedir informações específicas (como o ID da sessão) antes de responder perguntas sobre consumo ou tempo de recarga, em vez de dar uma explicação geral como fazia antes. Isso deixa a resposta mais próxima de um sistema real de atendimento, mas muda a experiência de conversa em relação à Sprint 2.

3. Comparativo antes × depois

Aspecto	Sprint 1 e 2	Sprint 03
Arquitetura	Loop manual chamando a API diretamente	Agente orquestrado pelo OpenAI Agents SDK
Memória	Lista Python acumulando mensagens manualmente	SQLiteSession gerenciada pelo framework
Modelo utilizado	gpt-4o-mini	gpt-4o-mini (comparado com gpt-5-nano)
Guardrails de segurança	0	2 (entrada e saída)
Ferramentas (tools)	0	4
Teste de memória (3 turnos)	Não realizado	Realizado — adequado
Teste de prompt injection	Não realizado	Realizado — 3/3 tentativas bloqueadas

Antes:

Nas Sprints 1 e 2, o chatbot chamava a API da OpenAI direto, sem framework. A memória era só uma lista guardando as mensagens. O modelo era o gpt-4o-mini. Não tinha nenhuma proteção de segurança nem ferramentas extras. Também não tinha testes de memória nem de prompt injection.

Depois:

Na Sprint 03, o chatbot passou a usar o OpenAI Agents SDK. A memória agora é guardada pelo próprio framework. O gpt-4o-mini continuou sendo o modelo principal, mas dessa vez foi comparado com o gpt-5-nano antes de escolher. Foram criadas proteções de segurança pra bloquear tentativas de burlar o chatbot, e quatro ferramentas que ele pode usar pra consultar status, consumo, cobrança e abrir chamados. Agora também existem testes de memória e de prompt injection.

A nova arquitetura realmente melhorou o chatbot?

Sim. A memória ficou mais organizada, gerenciada pelo próprio framework. As proteções de segurança são um ajudaram bastante, já que antes não existia nada disso. Mas, por outro lado, o chatbot ficou um pouco menos natural em duas perguntas, porque passou a pedir dados específicos ao invés de responder de forma geral. E um

dos modelos testados (gpt-5-nano) teve um bloqueio errado em um dos testes, o que mostra que vale a pena testar mais de um modelo antes de escolher.

4. Problemas encontrados e soluções

Problema 1 — Guardrail de saída bloqueando uma resposta correta

Durante os testes de memória com o modelo gpt-5-nano, uma resposta legítima do chatbot foi bloqueada pelo guardrail de saída, que a classificou incorretamente como uma possível orientação de risco.

Alternativas consideradas:

- (a) remover o guardrail de saída, para evitar bloqueios indevidos;
- (b) manter como estava e aceitar o risco de bloqueios acontecerem sem tratamento;
- (c) manter o guardrail, tratar a exceção no código para não travar a execução, e documentar o comportamento como uma limitação conhecida.

Solução adotada: a opção (c).

O guardrail de saída é parte importante dos requisitos de segurança do desafio, então removê-lo não era uma opção adequada. Foi adicionado tratamento de exceção em todas as funções de teste, para que um bloqueio não interrompa a execução do notebook, e o comportamento foi documentado no relatório de comparação de modelos.

Justificativa: manter uma camada de segurança, mesmo que as vezes imperfeita, é mais importante do que eliminar qualquer chance de bloqueio.

Problema 2 — Acesso restrito a modelos na conta da OpenAI

A conta da OpenAI usada não tinha acesso ao gpt-4o, impedindo testar esse modelo como planejado inicialmente para a comparação

- (a) solicitar/aguardar liberação de acesso na conta OpenAI;
- (b) usar um modelo mockado só pra simular a comparação;
- (c) consultar os modelos realmente disponíveis na conta e escolher outro modelo de chat válido

Solução adotada: opção (c): foi consultada a lista de modelos disponíveis (`client.models.list()`) e escolhido o gpt-5-nano, que estava acessível

Justificativa: manter a comparação com dados reais (mesmo que com outro modelo) é mais confiável do que simular resultados, e o gpt-5-nano ainda cumpria o requisito de ser um modelo diferente do usado como base

5. Divisão da equipe

Nosso grupo divide os trabalhos da disciplina por integrante, um trabalho inteiro para cada um,. A Sprint 03 ficou sob responsabilidade de Lara Dos Santos Cândido Alves, que desenvolveu o código, os testes e a documentação apresentados neste relatório.

Integrantes do grupo:

- Andrey Crence Fernandes — RM 573840
- Giulliana Maistro Brasolin — RM 569381
- Mikaella Mirela Dos Santos Lucindo — RM 573775
- Lana Namie Kano Ozeki — RM 569795
- Lara Dos Santos Cândido Alves — RM 573827 — responsável pelo desenvolvimento da Sprint 03
- Lucas Parkin Devito — RM 573251